

CSCE 632 ACCESSIBLE COMPUTING · MODULE 08 · LECTURE NOTES

Semantics and the Accessibility Tree

The mechanism, from which most of the rules follow.

MODULE OUTCOMES

- Explain the accessibility tree and how it is derived.
- Explain the name-role-value-state model.
- State the rules of ARIA and why the first one exists.
- Explain live regions and announcing change.
- Derive a rule from the mechanism rather than recalling it.

1 The tree

AN ASSISTIVE TECHNOLOGY READS A SEPARATE TREE THAT YOUR MARKUP GENERATES, NOT YOUR MARKUP AND NOT YOUR PIXELS

The platform builds an accessibility tree from your document — a parallel structure of nodes, each carrying a role, an accessible name, a value, and a set of states — and the assistive technology reads that tree through a platform accessibility API, never the DOM directly and never the rendered pixels.

Which means your visual design is entirely invisible to it and your semantics are everything — a div with a click handler and a background image produces a node with no role and no name, however convincing it looks, and is announced as nothing at all.

And nodes can be absent from the tree: images marked decorative, display: none content, visibility: hidden, and aria-hidden subtrees are all pruned — which is sometimes exactly what you want and is sometimes the bug, and the two cases look identical in the browser.

So the debugging question is always 'what is in the tree' — and every major browser's inspector will show you, which turns most accessibility debugging into reading a data structure rather than into guessing from a checklist.

OPEN THE ACCESSIBILITY PANE and for each control check the four things:

ROLE	what kind of thing is this? button, link, checkbox, heading, table... missing role => "generic", and unusable
NAME	what is it called? from its content, a label element, aria-label, aria-labelledby, or alt missing name => "button", unidentified
VALUE	what does it currently hold? the text entered, the option selected, the slider position
STATE	checked, expanded, selected, disabled, invalid, current, pressed, busy

THEN THE TREE SHAPE

is the heading outline right?
are the landmarks present?
is anything missing that should be there?
is anything there that should not be?

Role, name, value, state — for every control, and the accessibility pane in any browser shows all four beside the element. **This four-item check is the mechanical core of the course:** it is what a screen reader announces, it is what voice control matches against, and it is what the automated checkers are actually inspecting. Doing it while you build a control rather than afterwards is the habit this module is trying to install (§4's closing callout).

2 Native first

A NATIVE BUTTON IS A ROLE, A NAME, KEYBOARD OPERABILITY, FOCUS, AND STATE, FOR FREE

A button element gives you the button role, an accessible name computed from its content, activation on both Enter and Space, focusability in the right tab position, a platform focus indicator, and the disabled state — all of it, in every browser and on every platform, maintained by somebody else.

A div with a click handler gives you none of it — and reproducing all of it correctly takes a role, a tabindex, two separate key handlers, a focus style, and explicit state management, each of which can be wrong and one of which usually is.

So the rule is: use the native element, and the reason is that it is less code and more correct — which is an engineering argument rather than a compliance one, and is therefore the version to make in a code review.

And this extends everywhere: native form controls, the native dialog element, details, real table markup with header cells, fieldset and legend — each carries a keyboard model and a semantic contract you would otherwise owe and would have to keep in sync forever.

3 ARIA

- **1 · Do not use ARIA if a native element will do — no ARIA is better than bad ARIA, and bad ARIA is actively worse than none** because it asserts something false about the control (see the callout).
- **2 · Do not change native semantics unless you really must — giving a heading a button role removes it from the heading outline**, which removes it from the navigation (Module 03 §2).
- **3 · Every interactive ARIA control must be keyboard operable**, with the keyboard model its role implies — **a role does not bring behaviour with it, which is the commonest misunderstanding of what ARIA is**.
- **4 · Do not put `role="presentation"` or `aria-hidden="true"` on a focusable element — which produces a control that can be focused and cannot be identified**, and is a particularly confusing failure.
- **5 · Every interactive element needs an accessible name — and the icon-only button is where this is missed, universally. ARIA changes what is announced and never what anything does — so a role without the matching keyboard behaviour is a lie the user acts on.**

BECAUSE ARIA IS A PROMISE ABOUT BEHAVIOUR THAT YOU THEN HAVE TO KEEP

Adding `role="checkbox"` tells the user that Space will toggle this control and that its checked state will be announced — and the browser does not implement either of those for you: the role is a declaration, not an implementation.

So the user is told what to expect and the expectation fails — which is worse than an unlabelled element, because an unidentified control invites caution and a mislabelled one invites confidence, and acting confidently on false information is how data gets lost.

Which is the real content of the first rule: the native element keeps the promise and your ARIA merely makes it — and those are not symmetric amounts of work, which is the whole argument in one sentence.

And it is a general pattern worth noticing — a declaration that has to be kept manually synchronised with behaviour will drift — which is CSCE 713 Module 01's make-the-defect-unexpressible argument applied here: the native element makes the desynchronisation impossible to express.

4 Change

A SERIAL USER DOES NOT PERCEIVE CHANGE UNLESS IT IS ANNOUNCED

A search result count updating, a validation error appearing beneath a field, a toast notification, an item added to a cart, a progress bar advancing — all visible, none announced, and the user continues entirely unaware (Module 03 §2's serial channel with a cursor).

So a region that updates must be declared a live region, with a politeness level: **polite waits for a pause in speech, assertive interrupts immediately — and assertive is almost always the wrong choice**, because interrupting somebody mid-sentence loses their place.

And focus management is the other half of this: opening a dialog moves focus into it, closing it returns focus to the control that opened it, and route changes in a single-page application need both an explicit announcement and a deliberate focus move, since no page load happened to do it for you.

Which is the one area where a framework will not save you — because the framework does not know which of its re-renders are meaningful to a user and which are incidental, and only you do. This is the defect class to look for first in any application that does not reload pages.

AND THIS IS WHY THE MECHANISM IS WORTH THE HOUR IT TAKES

Once you know what the accessibility tree contains, most of the rules become derivable rather than memorised — 'why does an icon-only button need a label' has an answer rather than a citation — which is CSCE 671 Module 05's mechanism-over-rule argument and the same payoff.

And the four-item check generalises to everything: role, name, value, state, on every control — which is a complete audit procedure for the semantic half of this course and needs no checklist to remember.

Plus it is directly inspectable — the tree is right there in the browser's developer tools — which makes this the most debuggable part of accessibility and the part people nonetheless guess at most often.

So the habit to build is: open the accessibility pane while you are building the control, not after it is finished — **because the four items are a specification and not a test**, and treating them as a specification is what makes the repair work in Project 2 unnecessary next time.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
The ARIA Authoring Practices Guide (free)	§§2 and 3 — patterns with their required keyboard models, which is the part the role alone does not give you.
Using ARIA: the five rules (free)	§3's list in the original, with the reasoning for each and the first rule stated unambiguously.

Resource	What it covers for this module
The Accessible Name and Description Computation (free)	§1's name derivation — the precedence order, which explains surprising announcements.
The HTML Accessibility API Mappings (free)	§2's 'for free' , specified — exactly what each native element contributes to the tree.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Open the accessibility pane** on three pages and read the tree.
- 2 **Record role, name, value, and state** for ten controls in your own project.
- 3 **Find a control with a generic role** and fix it natively.
- 4 **Build a button twice** — natively, and from a div — and compare the line counts and the trees.
- 5 **Find an aria-hidden that hides something focusable.**
- 6 **Add role="checkbox" to a div** and observe what does and does not happen.
- 7 **Audit every icon-only button** in your project for an accessible name.
- 8 **Find three unannounced changes** in an application you use daily.
- 9 **Add a polite live region** to one of them and test it with a screen reader.
- 10 **Open and close a dialog** and verify focus returns correctly.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does an assistive technology actually read?
- 2 Name the four properties of a node.
- 3 When are nodes pruned from the tree, and why is that ambiguous?
- 4 List what a native button provides.
- 5 Why is 'use the native element' an engineering argument?
- 6 State the five rules of ARIA.
- 7 Why is bad ARIA worse than none?
- 8 What does ARIA change and what does it not?
- 9 Why does a serial user miss change, and what are the two fixes?

10 Why can a framework not handle live regions for you?